EtherBench Documentation

None

Leonard HEYWANG / EtherBench Open Hardware

Table of contents

1. Introduction

1.1 What's EtherBench ?

Etherbench is a platform, built arround a lot of different projects, designed to make debugging and testing easier, without requiring costly tools.

While it provides basic debugger features, according to the ARM CMSIS-DAP specification, over the two standard buses:

- SWD : Serial Wire Debug, common for MCUs
- JTAG : Joint Test Action Group, common for FPGAs and MPUs.

It also provides standard buses, to perform most of the IO a chip can do, within a single device. This include:

- 2 x USART buses, up to 20 Mbaud. One of them include the modem features (RTS and CTS signals).
- 1 x SPI bus, up to 50 Mbaud.
- 1 x I2C bus, up to 4 MHz.
- 1x CAN bus

And, some very basic analog features:

- 2 x Analog inputs, up to 16 bits (emulated).
- 1 x Analog Output.

All of this stuff is isolated from the host, thus even in case of failure on the DUT side, your PC remains safe.

To interract with the device, multiple options are offered:

- 100 Mbps Ethernet port (including DHCP)
- 12 Mbps USB port (behind a 480 Mbps hub, to wire others devices !).

Both options are similar in terms of features.

1.2 Why ?

I've built this project, because I was tired to accumulating probes on my desk. One for the ST's, one for the FPGA, and so on. Therefore, I've looked into different projects, such as this one. But, it wasn't complete, and didn't like the complete implementation, so, I've just rewrote my own implementation, with the latest features available.

The project then started as, and what if I add a USB-USART bridge to it ? And, wait, can't I also add an Ethernet port ? Now, we're here, reading the project documentation...

1.3 Highlights

All of those aspects are detailed within their associated pages. Here a rapid tour of the horizon:

- High performance hardware
- STM32H5 MCU, providing all the features of the probe
- Isolated PCB, ensuring safety of your designs.
- Broad range of communications
- Ethernet
- USB
- SDMMC
- Easy to get hands on
- Comprehensive leds signals
- Standard interfaces

1.4 More infos ?

There's a lot that isn't documented here. Please check all the subsections, or download the PDF document !

- Installing the device
- Hardware schematic and details
- Programming guides
- Examples
- Software architecture

2. Installing

2.1 Installing

As the project is based over three software components, installing can't be done with a single installer. Only the Firmware is required for the device to be working, all other are add-ons.

2.1.1 Embedded Firmware

1. You'll need a copy of the embedded firmware. The later one can be obtained from two sources:
   - Downloading a built firmware, on the releases.
   - Building your own, with the STM32Cube environnement.

   For most user, I recommend the first option.
2. Then, we need to program the chip on the board. Any tool can be used, such as CubeProgrammer, which can use the USB link to upload the firmware.

   Alternatively, there's SWD pins available, thus, any USB-SWD probes can be reused.

1. Once done, the board shall come alive. If that's the case, you'r done for the firmware.

2.1.2 Desktop Application

1. You'll need the installer, for the right exploitation system. You can download them on the releases. pages.
2. Run the installer.
3. Done

Alternatively, you can build the software from sources, over the repo.

2.1.3 Software library

The software library is actually a single header file, that provide abilities for the tool to send data in a known format to the App. This expand the abilities of the App without costing that much.

There's no installation, as you copy that file into your project, include it, and build with your usual toolchain. That's all !

2.2 Configuring

3. Hardware

3.1 Hardware

EtherBench, at least the probe is designed around an assembly of three PCB :

- One master board, with six layers. It host all the complex functions of the device.
- One top board, with the leds and basic leds features.
- Two lateral boards, with buttons and the screen.

The two later ones are done with a two layer boards, as there isn't any high speed signals nor complex requirements. Therefore, the cost of the device remains accessible.

The electrical part fit within a 110 x 110 mm boards assembly.

3.1.1 Master board

The master board is designed around

Note While the global concept of the boards is done, the specific details aren't. Thus, no boards / assemblies / schematic are going to be added. They'll be once finished.

3.1.2 Top board

The top board act as a user interface more than a functionnality board. it comports:

- A ring led, as the main interface with the user
- Some status leds

Led ring

The led rings, based over 20 WS2812 leds show the user animations, and statuses. This replace a complex gui with simpler animations. The GUI is then sent back over buses to the host computer.

Multiple animations can be shown :

Animation	Color	Description	Meaning
Cylon / Radar Spin	`#22F2D6`	A single bright Blue pixel spins around the 10-LED circle rapidly, leaving a fading trail of 3 pixels behind it.	Power on until RTOS threads are fully initialized and IP address is acquired.
Breathing	`#FDFF91`	All 10 LEDs slowly pulse on and off simultaneously using a sine wave function.	RTOS is idle, no active network connections, no target board connected.
Solid Activity	`#FFBF00`	The ring show a progress status of the flash interface. Restart from 0 when in debugging session.	Software has connected and claimed the USB interface.
Blink Activity	`#C9006F`	All 10 LEDs blink on and off rapidly at 5Hz (100ms on, 100ms off). Only for the device error (not the DUT).	DUT does not respond anymore (crash ?)
Progress bar	Yellow : `#E6DC20` or Violet : `#601AED` / `#9B71F0` / `#C6B1F2`	The ring show a progress status of the sequence.	User start a sequence. Yellow for custom one, Violet if triggered from the actions buttons.
Breathing	Green : `#34BA4A` or Red : `#C4314C`	All 10 LEDs slowly pulse on and off simultaneously using a sine wave function.	Sequence has ended. Here the results.
Strobe	`#E02926`	All 10 LEDs blink on and off rapidly at 5Hz (100ms on, 100ms off). Only for the device error (not the DUT).	HardFault, Ethernet disconnect, or Storage failure.

Leds

In addition of the main ring leds, there's 8 leds placed on the side. These show the user informations about what's being currently transfered.

Leds	What it means
Ethernet activity (Green / Yellow)	There's currently Ethernet trafic
Usb activity (Blue / Red)	There's currently USB trafic
Flash activity (Orange / Green)	There's currently memory IO, especially to the SD card
Status (blink, green)	I'm alive !

3.1.3 Side boards

The side boards are used as user inputs, and thus contain essentially buttons. A small I2C screen is added, to display basic informations on it, such as the granted IP in case of the DHCP attribution.

For each EtherBench probe, there's two of these boards, on each side.

Feature	Right board	Left board
OLED screen	🚫	✅
Go button	🚫	✅
Stop button	🚫	✅
Reset	✅	🚫

OLED screen

The onboard screen is used to show specific infos, that can't be reported in any other ways. This include, as explained before the IP, because at this point, your not connected.

It's also used to show additionnal infos, such as:

- Sequences info (what failed ?)
- Error codes, to get more specific infos (rather than a generic red flash)
- Progressions
- Custom infos, from the sequence support.

Go / Stop button

These buttons are used as trigger for specific, sequence related actions.

Therefore, it's possible to start or stop a sequence that can be execute from within the flash, in total autonomy.

Reset button

The button will perform an hardreset of the device. That's why it's quite "hard"

3.2 Mechanical

Now, let's show all mechanical dimensions, for the case or more advanced integrations.

4. Programming

4.1 Programming

The device can be controlled from different ways, over different mediums:

Interface	USB	Ethernet
File transfer	None*	FTP server
USART Bridge	VCOM 1	None**
Debugging	USB Debugger probe, CMSIS-DAP compatible	Remote debugger probe, CMSIS-DAP compatible
Terminal	VCOM 0	Telnet Shell
SCPI	VCOM 0	Dedicated SCPI port

- * The file transfer can still be done by removing the SD card from the device. But, as a limitation from the filesystem we're using, files can't be copied from the flash to the SD.
- ** This feature can be emulated by the shell terminal. Some commands can perform IO, and thus this is possible. Speed may be affected.

4.2 Sequences

Sequences are a big part of the Etherbench project. A lot of steps require some form of sequencing. Currently, there air two options to address these requirements:

- Internal sequencing
- External sequencing

The first one is executed on the device, thereby providing the fastest responses. This is also the most limited option, as only very simple structures are possible. The latter one is running on any host computer. It's response times are dependent on the IO method used, and software stacks. Any algorithm can be implemented here.Any algorithm can be implemented here

4.2.1 Internal sequencing

The internal sequences only support five instructions:

Instruction	Syntax	Description	Example
`execute`	`execute <command>`	Execute the passed SCPI string, as it was passed by any IO.	`execute *IDN`
`if`	`if <string>`	Evaluate the condition, and, if true, execute the code next. Will stop when an `else` or `end` is found.	`if *IDN,OK`
`else`	`else`	End an `if` condition, and only evaluate if the first condition was false.	`else`
`end`	`end`	Any condition, regardless of the initial evaluation	`end`
`print`	`print <string>`	Print the passed string to the console.	`print Hello World !`
`increment`	`increment <target>`	Increment the target variable (pass or fail). Can be used to track success or fails.	`increment pass`

Variables

The internal sequence engine does not support user variables. There air only two of these : pass and fail. For a sequence to return 0 (success), `pass > 0` and `fail = 0`.

Structures

The internal sequence engine does only support comparison to the passed string, and the latest result. This one is stored within the internal memory of the interpreter.

> ⚡ **CAUTION**
>
> The internal engine does not support any control flow loops or branch jumps. If loops are required, you must use external sequencing.

4.2.2 External sequencing

The external sequencing come to address all the limitations of the internal sequencing. And, the good news is that they're complementary !

The external sequencing can be done with **any** tools that support SCPI or telnets commands, thus, Python with pyvisa, LabView, and others !

For the greatest implementation, you can use the Desktop App, which include a Python interpreter, as well as a pre-intergated C++ Python library, which does all the heavy lifting for you ! Remains the pleasure of typing commands, with the probe class (`probe.send("uart", 0x42)`). You can still use the standard library of your system, thus, numpy, matplotlib, and others !

And, as announced, you can use the `probe.run("flash.seq")` to run an internal sequence ! You're getting the best of both worlds : The ideal scripting ability of Python with the performance of custom sequences !

4.3 Markers

All along the project, different structures are used. One of the most important is the marker stored on the daughter-board, to identify the target we're working with.

This data must be stored on an I2C EEPROM at address 0x50 on the board. The page size used is 32 bytes, to match with any reference on the market. Our is ST M24C64-RMN6TP from ST, but any reference that match theses options are good. This chip is always powered with 3.3V, even when the main supply is OFF.

4.3.1 Principle

The markers are based over a minimal set of information, stored within 32 bytes of data, and some optionnal pages that may store more data. Thus, the protocol is extensible.

Optionnal pages are referenced within the first page.

4.3.2 Page 0

This main page store the basic element, that ALL extensions board must have. The data stored within is arranged as:

Offset	Size (B)	Description
0x00	2	Page 0 marker, constant to 0xEBB0
0x02	2	Page version, must be 0x1000
0x04	4	Hardware version. Interpreted as M.m.pp values.
0x08	2	Default asked voltage. Interpreted as XX mV.
0x0A	2	Maximal current requested. Interpreted as XX mA.
0x0C	2	Lowest possible voltage. Interpreted as XX mV.
0x0E	2	Highest possible voltage. Interpreted as XX mV.
0x10	2	Custom flags. See bottom definition.
0x12	2	Clock frequency. Interpreted as XX MHz.
0x14	4	Optionnal page 0
0x18	4	Optionnal page 1
0x1C	4	Config CRC

Flags

Flags are individual bits, that identify a specific option

Bit	Name	Description
0	Isolated board	Indicate that the board isolate it's ground relatively to the master ground.
1	Require clock	Indicate that the board require a clock to operate.
2	Required USB	Indicate that the board use the onboard USB over the PCIe slot.
3	Active board	Indicate that the board include active components on between the host and the IOs.
4	Reserved	/
5	Reserved	/
6	Non standard form factor	Indicate that the board as a non standard form factor.
7	Ignore mouting check	Indicate to ignore the mouting security bits. This feature must be also forced from the shell.

Optionnal pages

Optionnal pages can be passed to the structure, to extend the functions of the latter one. The structure is quite simple: All of these are designed to fit over a single four byte space.

Offset	Size	Description
0x00	2	Address of the page
0x02	2	Page type (0xEBXX)

If unused, let 0xFFFFFFFF as the page ID.

4.3.3 Optionnal pages

As optionnal pages, a lot can be passed to expand the abilities. Here the different structures that can be passed:

Name	Type	Description
Pages	0xEBB1	Store up to 8 optionnal pages pointers. A single entity of this is possible.
Peripheral config	0xEBB2	Store peripheral basic configuration, to ensure a direct operation. Usefull with specific active components on the board.
Board name	0xEBB3	Store the human name of the board
Calibration	0xEBB4	Store calibration values for the analog IOs

Pages

Offset	Size (B)	Description
0x00	2	Pages marker, constant to 0xEBB1
0x02	2	Page CRC
0x04	4	Optionnal page 0
0x08	4	Optionnal page 1
0x0C	4	Optionnal page 2
0x10	4	Optionnal page 3
0x14	4	Optionnal page 4
0x18	4	Optionnal page 5
0x1C	4	Optionnal page 6

Up to 7 pages can be added, maxing out at 8 optionnal pages in total.

Peripherals

Offset	Size (B)	Description
0x00	2	Peripheral config marker, constant to 0xEBB2
0x02	2	Page CRC
0x04	2	Enabled peripherals (see mask at the bottom)
0x06	1	GPIO Direction (4 LSB) : 1 : in; 0 : out
0x07	1	GPIO Value (4 LSB)
0x08	3	UART 0 Config
0x0B	3	UART 1 Config
0x0E	3	UART 2 Config
0x11	3	SPI Config
0x15	4	CAN Config
0x19	4	JTAG / SWD Config
0x1D	3	I2C Config

PERIPHERALS FLAGS

Bit	Name
0	UART 0
1	UART 1
2	UART 2
3	Reserved
4	I3C / I2C
5	SPI
6	CAN
7	JTAG
8	Reserved
9	GPIO 0
10	GPIO 1
11	GPIO 2
12	GPIO 3
13	Reserved
14	Reserved
15	Reserved

UART CONFIG

Offset	Size	Description
0x00	2	Speed : Baudrate / 100
0x02	1	Bits 0-1 : Transfer size : 7 + 2'bXX. Bits 3-4 : Stop bits number : 2'bXX (1 or 2). Bit 7 : Enable parity.

I2C CONFIG

Offset	Size	Description
0x00	1	Speed : Baudrate / 1000
0x01	1	Bit 0 : Enable slave mode. Bit 2 : Enable 10 bit address. Bits 7-5 : Slave address [10-8]
0x02	1	Slave address [7:0]

SPI CONFIG

Offset	Size	Description
0x00	2	Speed : Baudrate / 1000
0x02	1	Transfer size (4-32)

CAN CONFIG

Offset	Size	Description
0x00	1	???
0x01	1	???
0x00	2	???
0x01	3	???

JTAG / SWD CONFIG

Offset	Size	Description
0x00	2	Speed : Baudrate / 1000
0x02	2	Bit 0 : Connect under reset

Board name

Offset	Size (B)	Description
0x00	2	Pages marker, constant to 0xEBB3
0x02	2	Page CRC
0x04	28	ASCII coded name.

Calibration values

Offset	Size (B)	Description
0x00	2	Pages marker, constant to 0xEBB4
0x02	2	Page CRC
0x04	4	ADC 0 Offset (float)
0x08	4	ADC 1 Offset (float)
0x0C	4	ADC Gain (float)
0x10	4	DAC 0 Offset (float)
0x14	4	DAC 1 Offset (float)
0x18	4	DAC Gain (float)
0x1C	2	Calibration year
0x1E	1	Calibration month
0x1F	1	Calibration day

4.4 Programming reference:

4.4.1 *cls

- SCPI: *CLS

⚡ Very fast (<10µs) 🔒 Blocking

Clear all the SCPI exposed registers on the device.

Arguments / Returns

This command does not take any arguments

Returns	Description
`CLS`	OK

Additionnal infos

> ⚠️ **WARNING**
> This does not perform a reset of the device. Hence, statuses may be retained.

Examples

Command : `*cls`

Return : `*CLS,OK`

4.4.2 *ese

• SCPI: *ESE

⚡ Very fast (<10µs) 🔒 Blocking

Read / Set bits on the Event Status Register. Returns the event status mask.

Arguments / Returns

Argument	Description
`XX`	Bits to be set (if maskable).

Returns	Description
`0-255`	Applied mask
`-1`	Invalid mask

Additionnal infos

Bit signification

Bit	Name	Signification
0	OPC	Operation complete
1		Unused
2	QYE	The device could not process a command. Buffers are perhaps full ?
3	DDE	Self test failed
4	EXE	A command failed it's execution
5	CMD	An invalid command was received
6		Unused
7		Unused

Examples

Command : `*ese 255`

Return : `*ESE,255`

4.4.3 *esr

- SCPI: *ESR

⚡ Very fast (<10µs) 🔒 Blocking

Read the Event Status Register.

Arguments / Returns

This command does not take any arguments

Returns	Description
`XX`	255

Additionnal infos

Bit signification

Bit	Name	Signification
0	OPC	Operation complete
1		Unused
2	QYE	The device could not process a command. Buffers are perhaps full ?
3	DDE	Self test failed
4	EXE	A command failed it's execution
5	CMD	An invalid command was received
6		Unused
7		Unused

Examples

Command : `*esr?`

Return : `*ESR,25`

4.4.4 *idn

- SCPI: *IDN

⚡ Very fast (<10µs) 🔒 Blocking

Return the identification string for the device.

Arguments / Returns

This command does not take any arguments

Returns	Description
`IDN`	

Examples

Command : `*idn?`

Return : `*IDN,EtherBenchv1:[build-date]:[firmware-version]`

4.4.5 *opc

- SCPI: *OPC

⚡ Very fast (<10µs) 🔒 Blocking

Check for current running commands

Arguments / Returns

Argument	Description
`XX`	Value

Returns	Description
`0`	OK

Examples

Command : `*opc?`

Return : `*OPC,OK`

4.4.6 *opt

- SCPI: *OPT

⚡ Very fast (<10µs) 🔒 Blocking

Returns the options string of the device. Currently, no options are possibles.

Arguments / Returns

This command does not take any arguments

Returns	Description
`0`	OK

Examples

Command : `*opt?`

Return : `*OPT,EtherBenchv1:[]`

4.4.7 *rst

- SCPI: *RST

⚠️ Slow 🔒 Blocking

Reset the device, equivalent to a power on-off-on cycle.

Arguments / Returns

This command does not take any arguments

Returns	Description
`RST`	OK

Additionnal infos

> ⚠️ **WARNING**
> Perform an hard reset of the device. Any openned sessions will be *brutally* closed.

Examples

Command : `*rst`

Return : `*RST,OK`

4.4.8 *sre

- SCPI: *SRE

⚡ Very fast (<10µs) 🔒 Blocking

Return the service query status.

Arguments / Returns

This command does not take any arguments

Returns	Description
`0`	OK

Additionnal infos

> 📝 **NOTE**
> This command is implented as a placeholder. Always return false.

Examples

Command : `*sre?`

Return : `*SRE,0`

4.4.9 *stb

- SCPI: *STB

⚡ Very fast (<10µs) 🔒 Blocking

Read the status byte register

Arguments / Returns

This command does not take any arguments

Returns	Description
`STB`	OK

Additionnal infos

> 📝 **NOTE**
> This command is implented as a placeholder. Always return 0.

Examples

Command : `*stb?`

Return : `*STB,0`

4.4.10 *tst

- SCPI: *TST

⚡ Very fast (<10µs) 🔒 Blocking

Perform the self-test of the device. Check that all threads are running.

Arguments / Returns

This command does not take any arguments

Returns	Description
`TST`	OK

Examples

Command : `*tst`

Return : `*TST,OK`

4.4.11 *wai

- SCPI: *WAI

⚠️ Slow 🔒 Blocking

Wait for all IO to be executed.

Arguments / Returns

This command does not take any arguments

Returns	Description
`WAI`	OK

Examples

Command : `*wai`

Return : `*WAI,OK`

5. Examples

5.1 Examples

6. Architecure

6.1 Architecture